

Data Structures

Assoc. Prof. Dr. Hilal ARSLAN

Week 4: Sorting Cont.

Sorting Algorithms

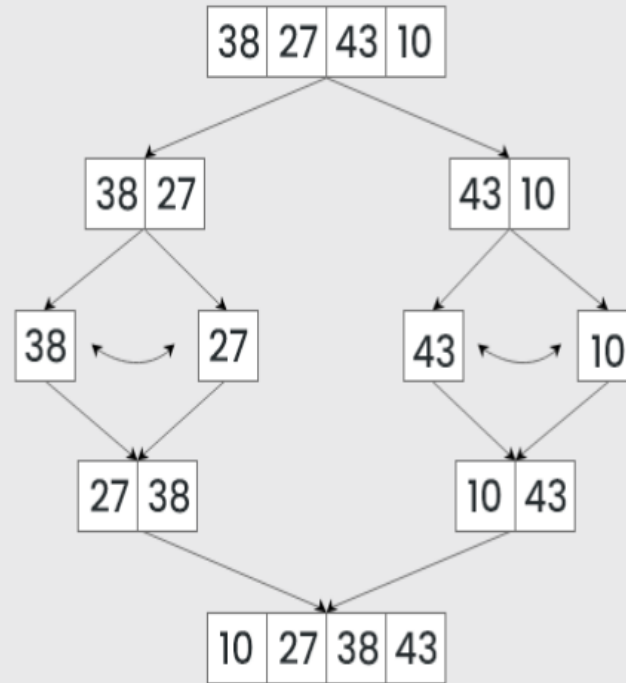
- There are many sorting algorithms, such as:
 - Selection Sort
 - Bubble Sort
 - Counting Sort
 - Radix Sort
 - Merge Sort
 - Quick Sort

5. MERGE SORT

- Merge sort algorithm is **divide-and-conquer** sorting algorithms
- It is a recursive algorithm.
 - Divides the list into halves,
 - Sort each half separately, and
 - Then merge the sorted halves into one sorted array.

Merge Sort

Algorithm



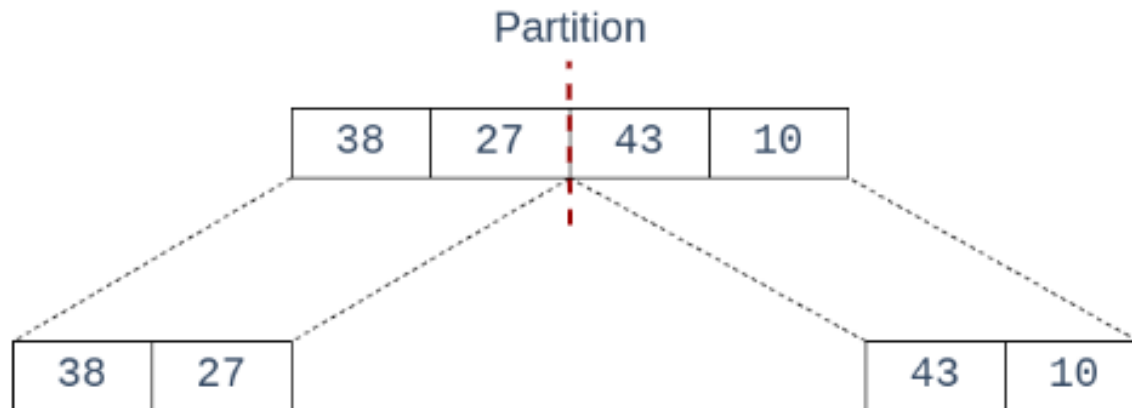
- Merge sort is a **recursive algorithm** that continuously splits the array in half until it cannot be further divided i.e.,
 - the array has only one element left
 - an array with one element is always sorted
- Then the sorted subarrays are merged into one sorted array.

Merge Sort

- Lets consider an array `arr[] = {38, 27, 43, 10}`
- Initially divide the array into two equal halves:

STEP
01

Splitting the Array into two equal halves

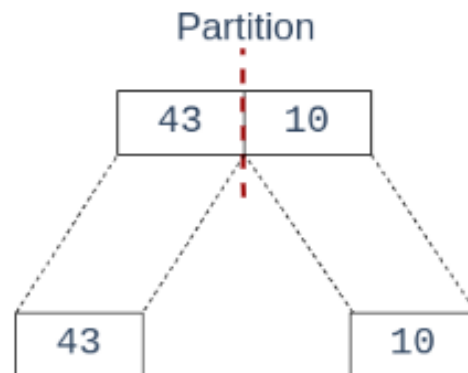
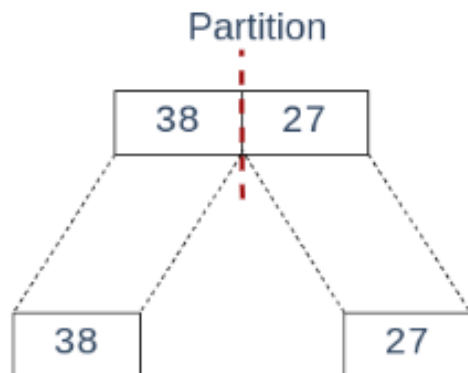


Merge Sort

- These subarrays are further divided into two halves. Now they become array of unit length that can no longer be divided and array of unit length are always sorted.

STEP
02

Splitting the subarrays into two halves

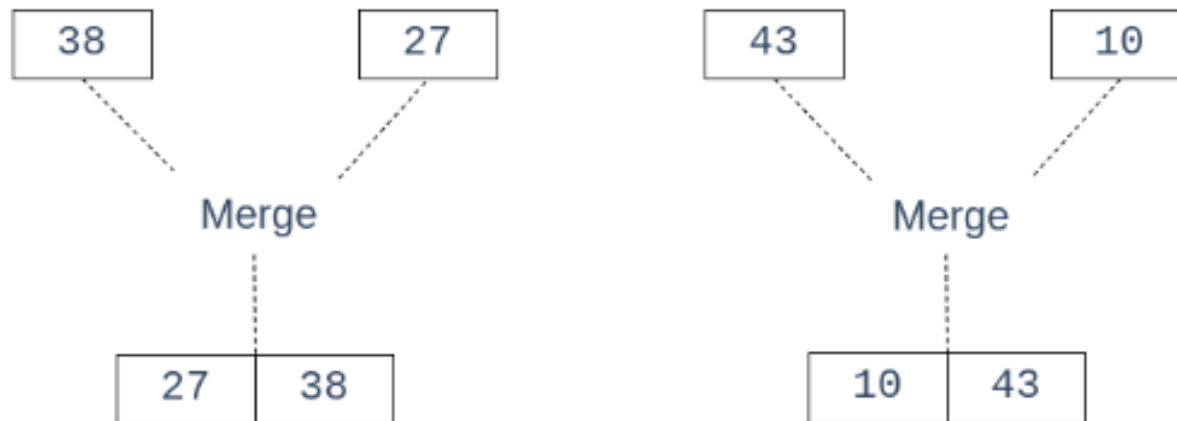


Merge Sort

- These sorted subarrays are **merged together**, and we get bigger sorted subarrays.

STEP
03

Merging unit length cells into sorted subarrays

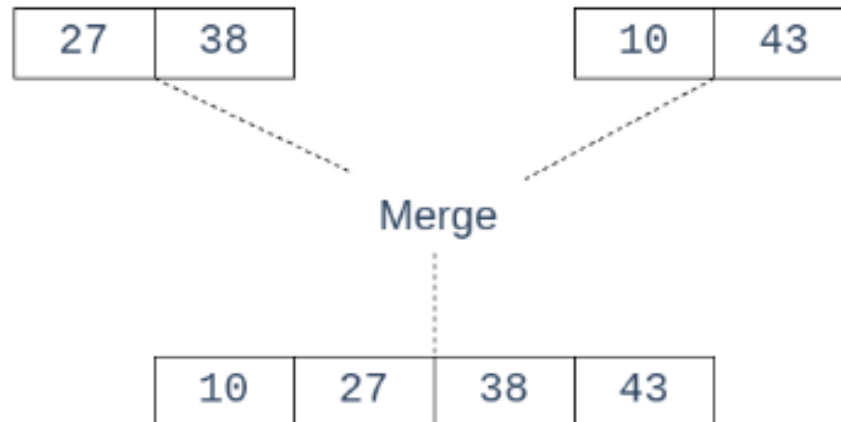


Merge Sort

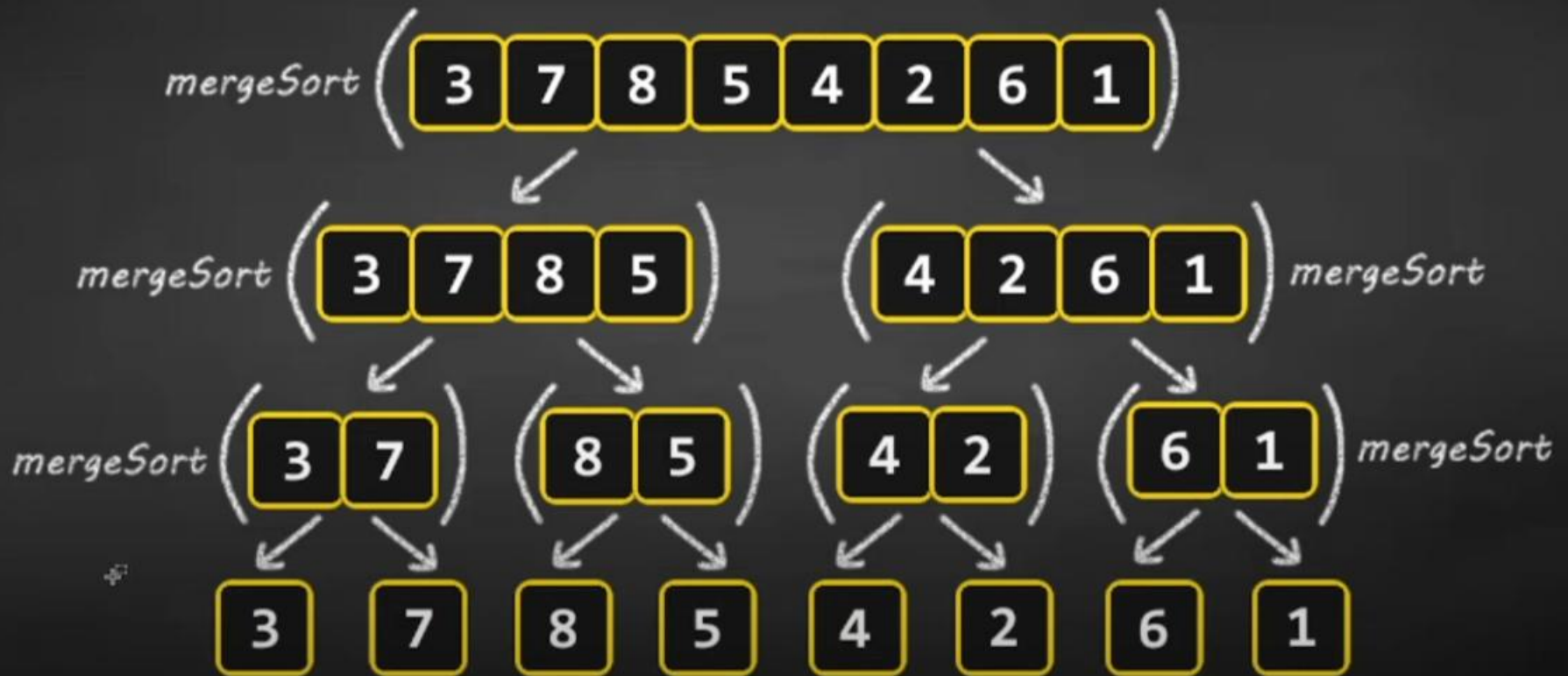
- This merging process is continued until the sorted array is built from the smaller subarrays.

STEP
04

Merging sorted subarrays into the sorted array



MergeSort



MergeSort

3 7 8 5 4 2 6 1

3 7 8 5

4 2 6 1

3 7

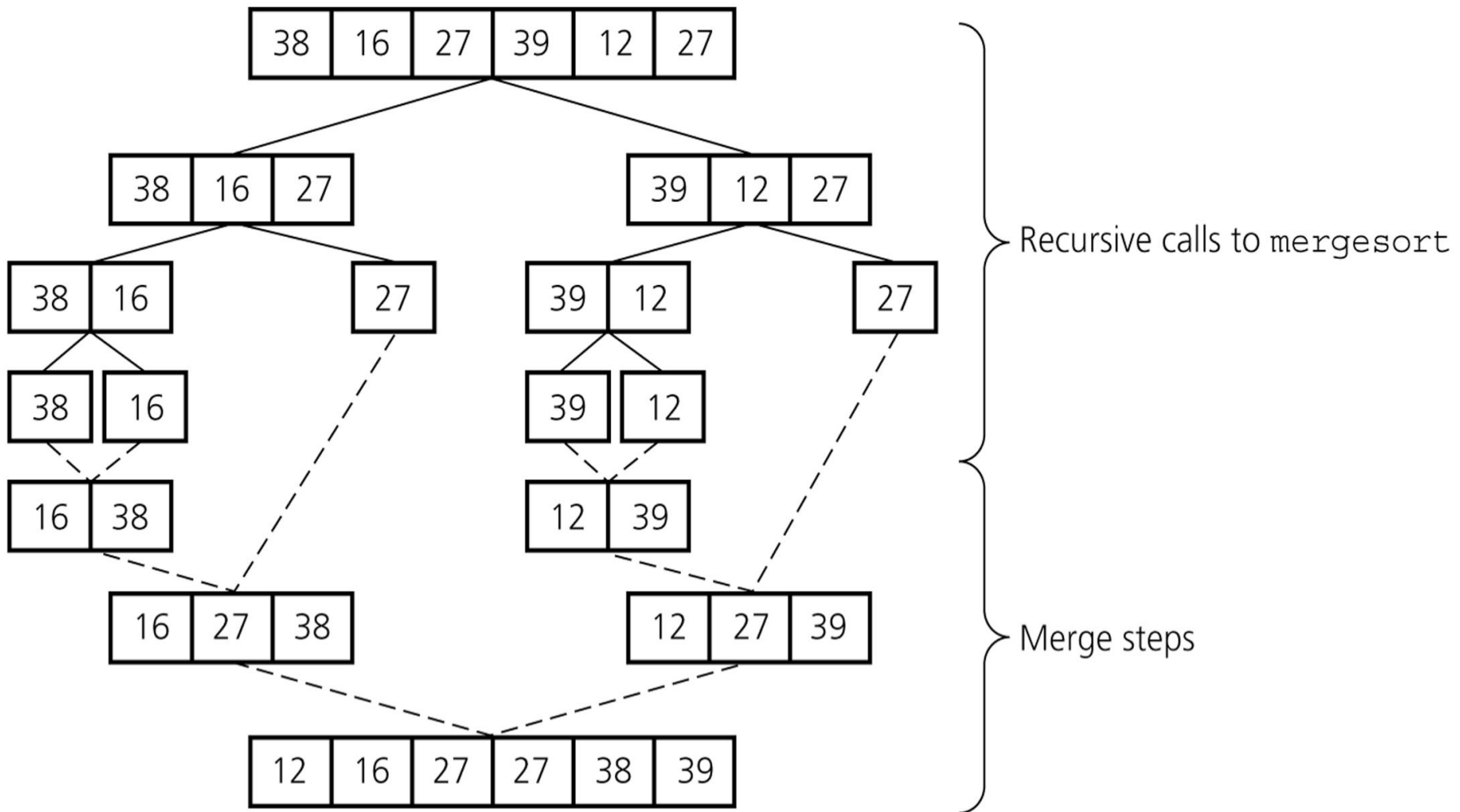
5 8

2 4

1 6

merge (3 7) (8 5) (4 2) (6 1) merge

Mergesort – Example



Merge Sort Example

2	8	5	3	9	4	1	7
---	---	---	---	---	---	---	---

2	8	5	3
---	---	---	---

9	4	1	7
---	---	---	---

2	8
---	---

5	3
---	---

9	4
---	---

1	7
---	---

2

8

5

3

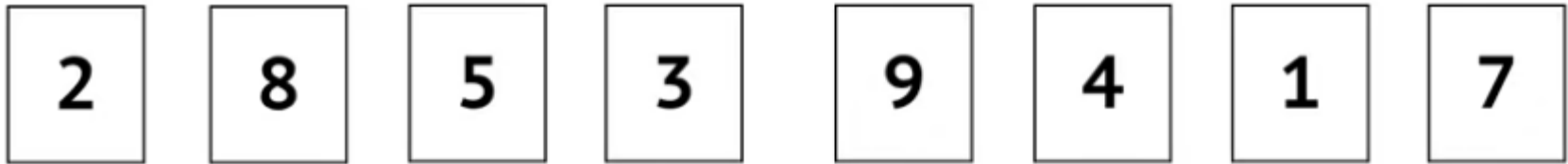
9

4

1

7

Merge Sort Example



MERGE-SORT(A, p, r)

1 **if** $p < r$

2 **then** $q \leftarrow \lfloor (p + r)/2 \rfloor$

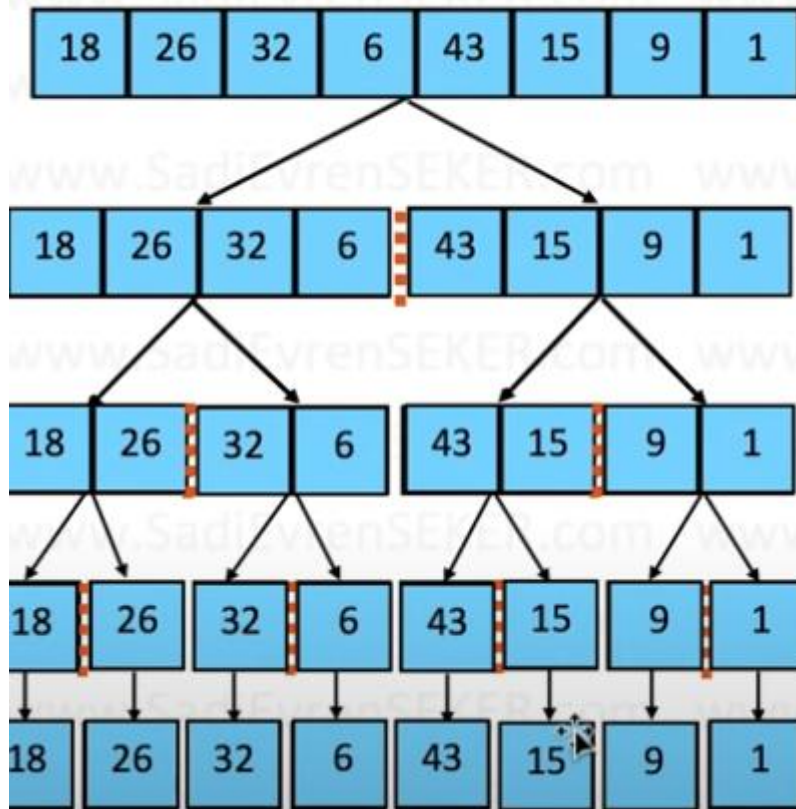
3 MERGE-SORT(A, p, q)

4 MERGE-SORT($A, q + 1, r$)

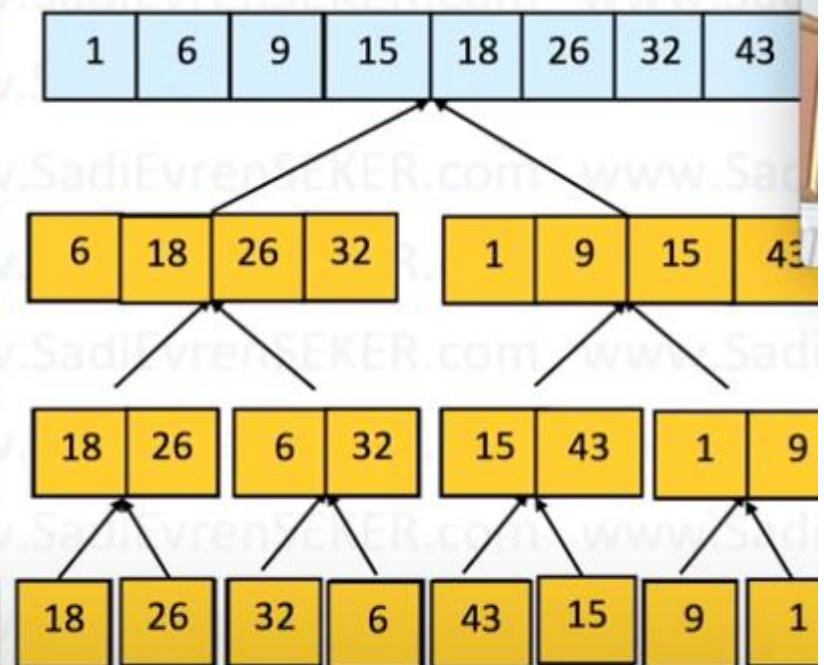
5 MERGE(A, p, q, r)

Initial call: MERGE-SORT($A, 1, n$)

Original



Sorted



```
void mergeSort(int arr[], int l, int r)
{
    if (l < r) {
        int m = l + (r - l) / 2;

        // Sort first and second halves
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        merge(arr, l, m, r);
    }
}
```



```

// Merges two subarrays of arr[].
// First subarray is arr[l..m]
// Second subarray is arr[m+1..r]
void merge(int arr[], int l, int m, int r)
{
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    // Create temp arrays
    int L[n1], R[n2];

    // Copy data to temp arrays L[] and R[]
    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    // Merge the temp arrays back into arr[l..r]
    i = 0;
    j = 0;
    k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
}

```

```

// Copy the remaining elements of L[],
// if there are any
while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

// Copy the remaining elements of R[],
// if there are any
while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}

```

```

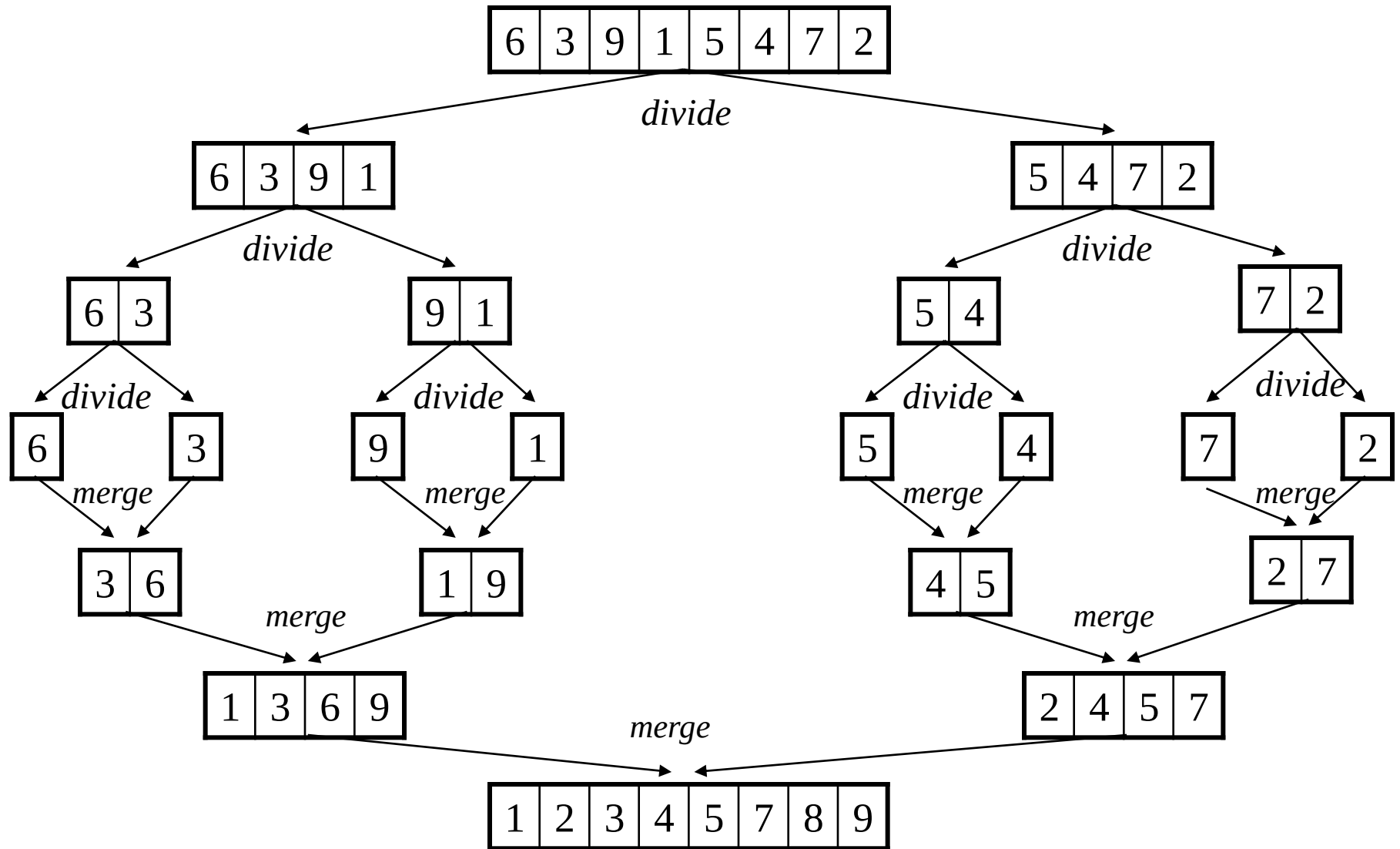
}

```

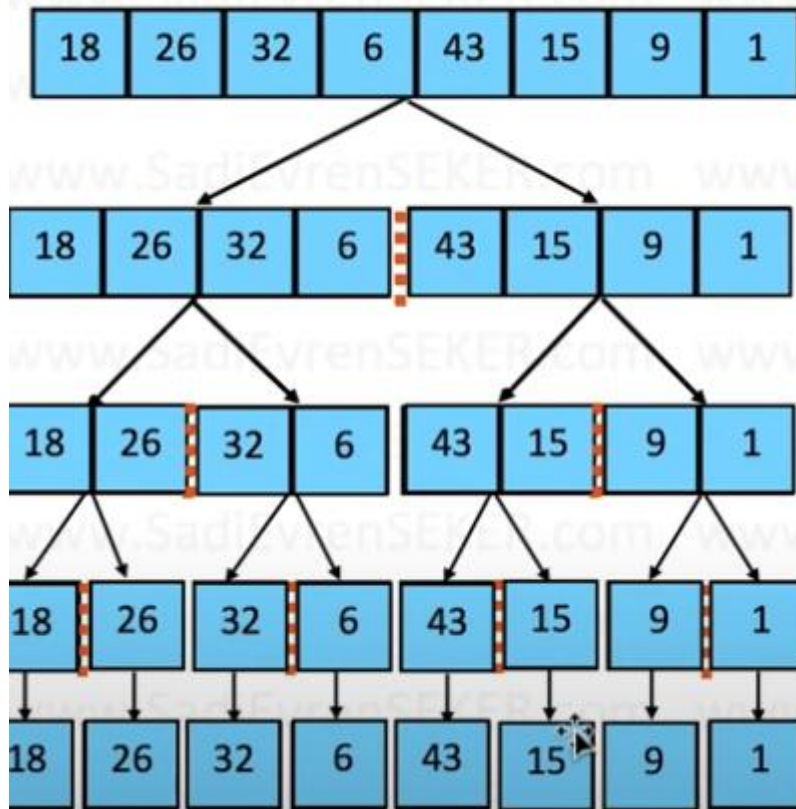
Question: Merge Sort

6	3	9	1	5	4	7	2
---	---	---	---	---	---	---	---

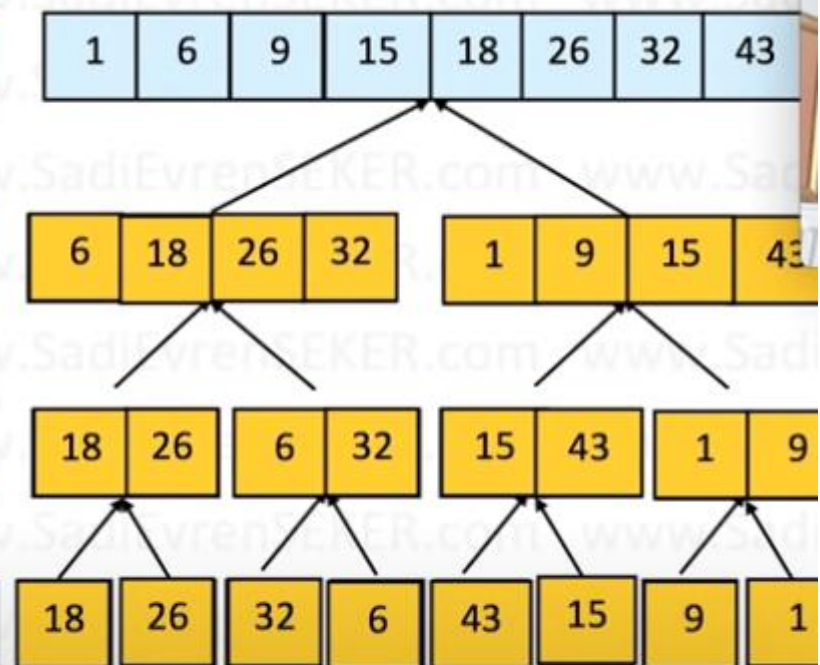
Mergesort - Example



Original



Sorted



Complexity $O(n \lg n)$

MERGE-SORT(A, p, r)

1 **if** $p < r$

2 **then** $q \leftarrow \lfloor (p + r) / 2 \rfloor$

3 MERGE-SORT(A, p, q)

4 MERGE-SORT($A, q + 1, r$)

5 MERGE(A, p, q, r)

6. QUICK SORT

Quick Sort

- A popular and fast sorting algorithm.
- Proposed by Hoare in 1962.
- Divide-and-conquer algorithm.
- Very practical
 - Most C++ implementations of “sort” use a combination of quick sort with other sorting algorithms.
- On average, it runs in time $\Theta(n \log n)$.
 - In fact, this happens 99.9% of the time.
- In the worst case, it runs in time $\Theta(n^2)$.
 - However this happens with astronomically small probability.

Quick Sort

- Based on **Divide and Conquer** algorithm
- Picks an element as a pivot
 - Put all smaller elements to the left of the pivot, and
 - all greater elements to the right of the pivot
- Partition is done recursively on each side of the pivot after **the pivot is placed in its correct position** and this finally sorts the array.

Divide and Conquer

Quicksort an n-element array:

1. Divide: Partition the array into two subarrays around a pivot x such that elements in lower subarray $\leq x \leq$ elements in upper subarray.



2. Conquer: Recursively sort the two subarrays.

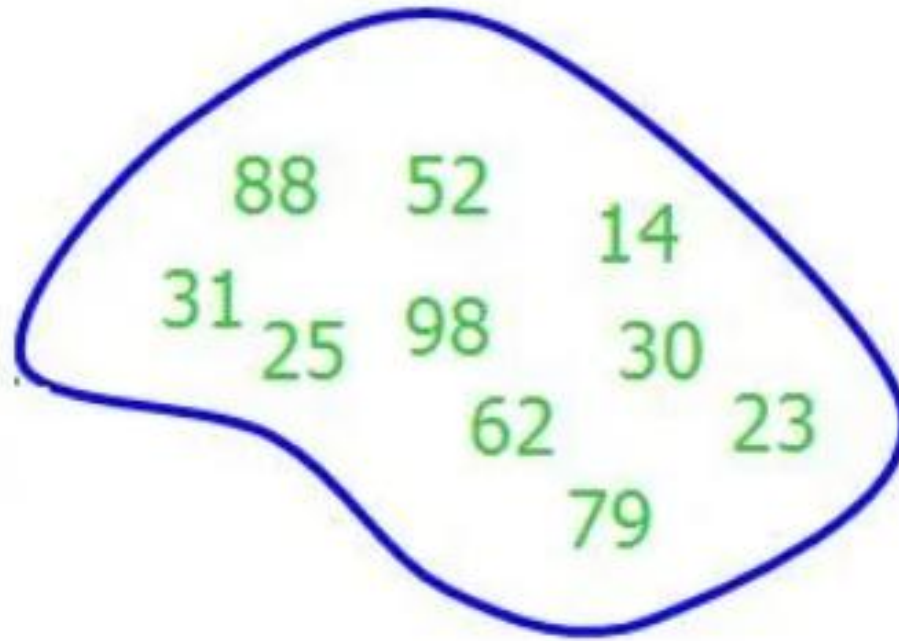
3. Combine

Pivot Selection

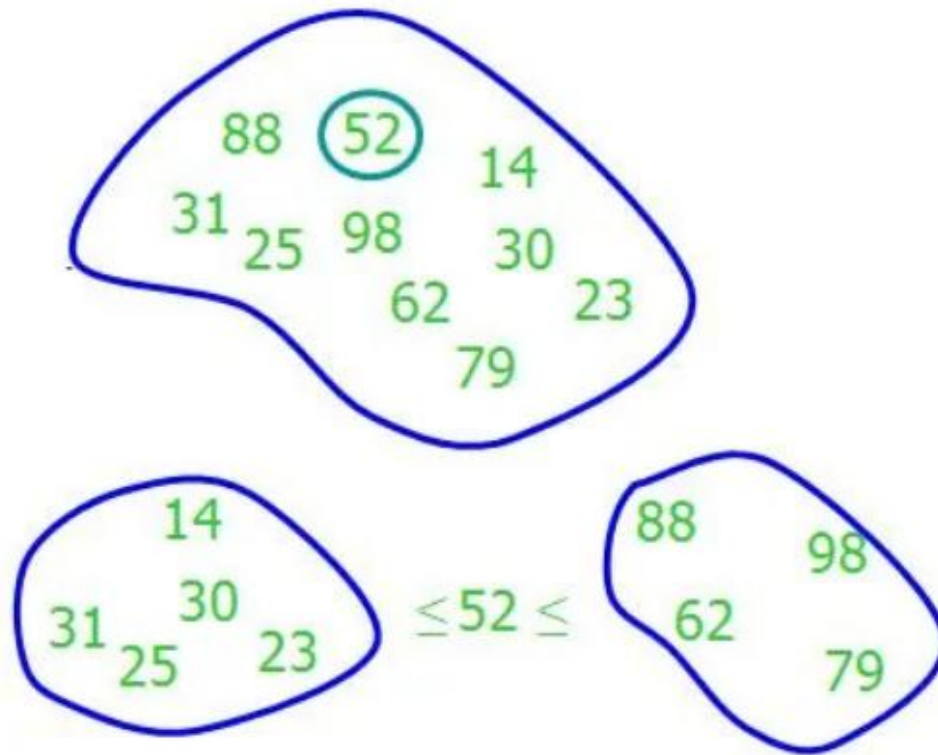
- Pick the last element as a pivot
- Pick the first element as a pivot
- Pick a random element as a pivot
- Pick the middle as the pivot.

Quick Sort

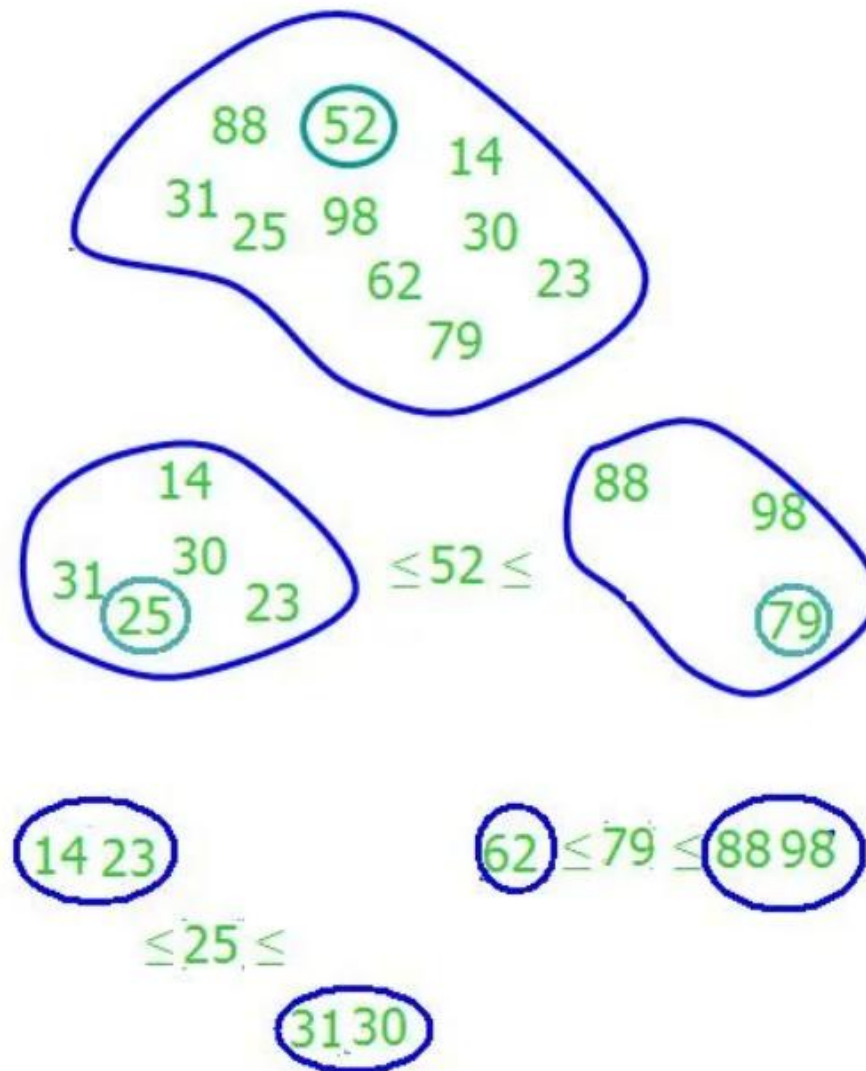
- Numbers: 88,31,52,25,98,14,30,62,23,79



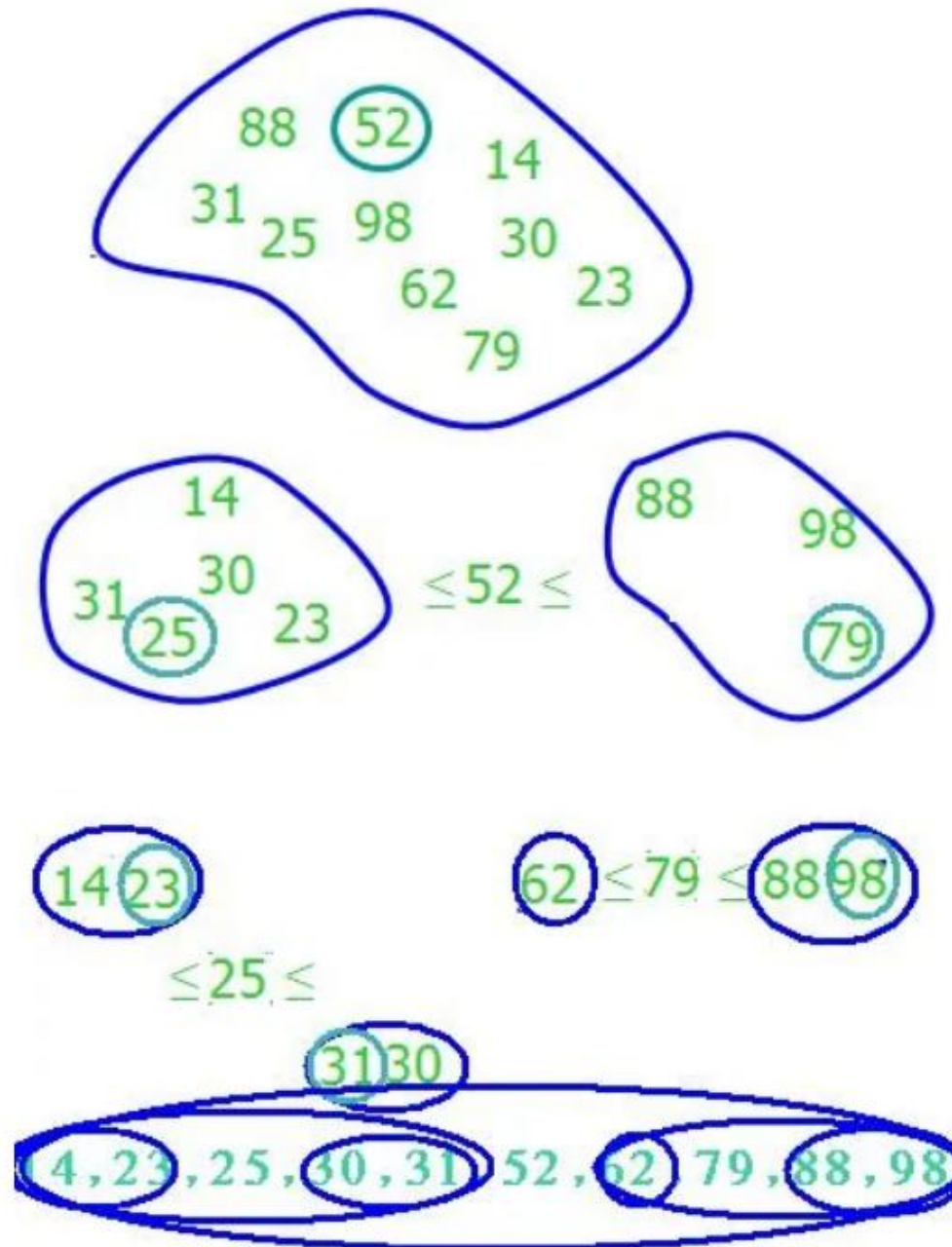
Quick Sort



Quick Sort



Quick Sort



Pseudocode for quicksort

QUICKSORT(A, p, r)

if $p < r$

then $q \leftarrow \text{PARTITION}(A, p, r)$

 QUICKSORT($A, p, q-1$)

 QUICKSORT($A, q+1, r$)

Initial call: QUICKSORT($A, 1, n$)

Quick Sort:

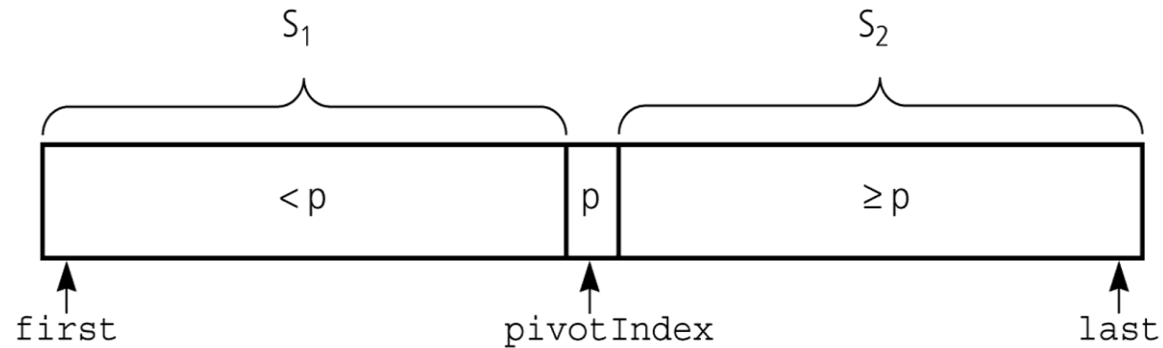
9	2	7	3	0	6	4
---	---	---	---	---	---	---

There are two important terms

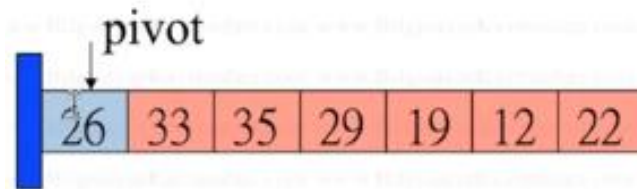
- Pivot
- Partition
 - Locate pivot at the correct position
 - Smaller elements at the left of pivot
 - Larger elements at the right of pivot
- Apply Partition to left and right part

0	2	3	4	6	9	7
---	---	---	---	---	---	---

- Partitioning places the pivot in its correct place position within the array.

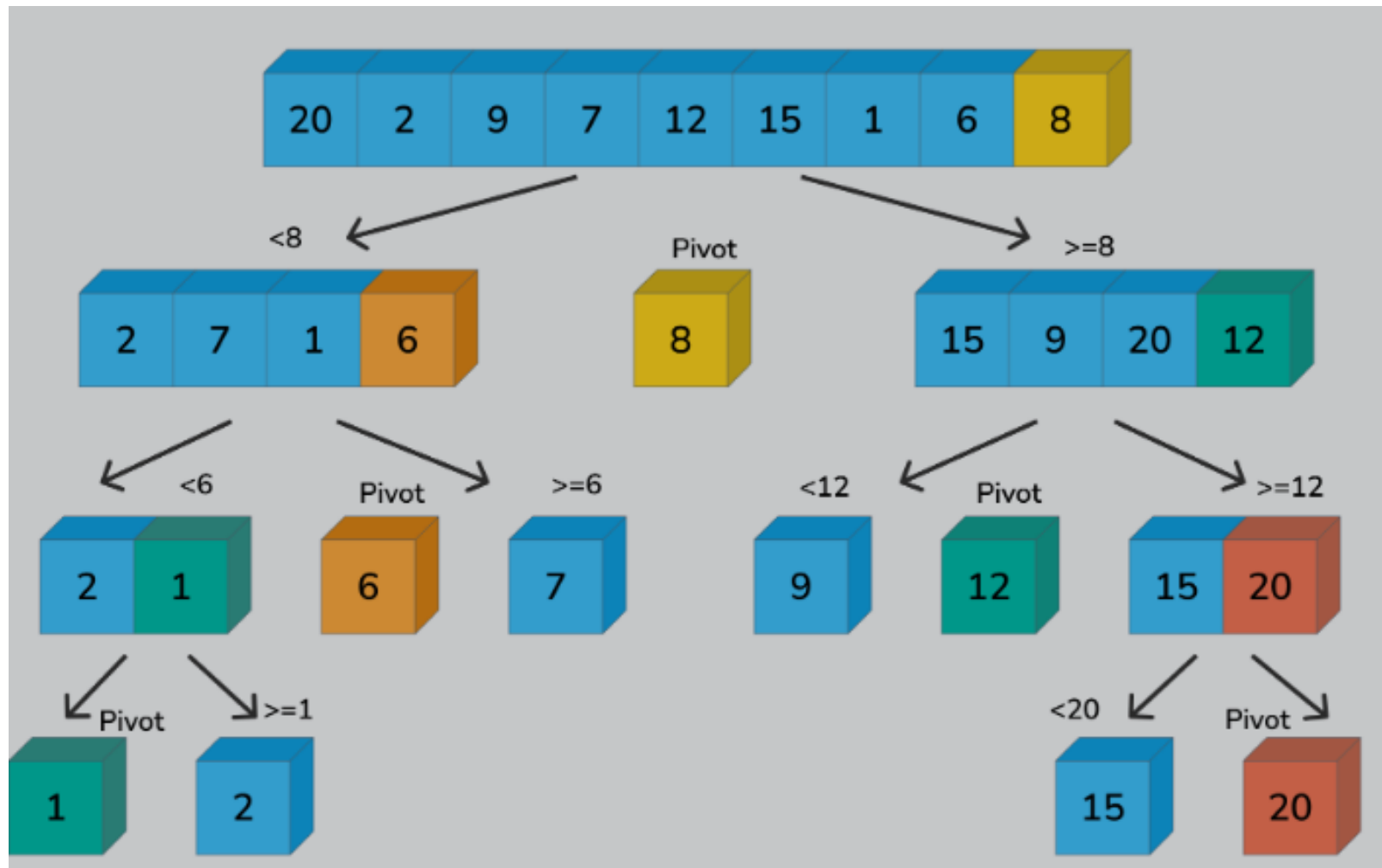


- Arranging the array elements around the pivot p generates two smaller sorting problems.
 - sort the left section of the array, and sort the right section of the array.
 - when these **two smaller sorting problems are solved recursively**, our bigger sorting problem is solved.



22,12,19 26, 33,35,29

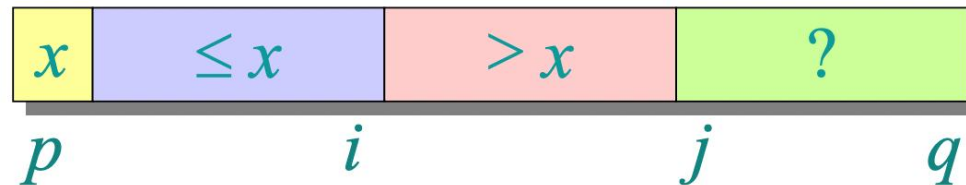
Example



Partitioning subroutine

```
PARTITION( $A, p, q$ )  $\triangleright A[p \dots q]$   
   $x \leftarrow A[p]$   $\triangleright$  pivot =  $A[p]$   
   $i \leftarrow p$   
  for  $j \leftarrow p + 1$  to  $q$   
    do if  $A[j] \leq x$   
      then  $i \leftarrow i + 1$   
           exchange  $A[i] \leftrightarrow A[j]$   
  exchange  $A[p] \leftrightarrow A[i]$   
  return  $i$ 
```

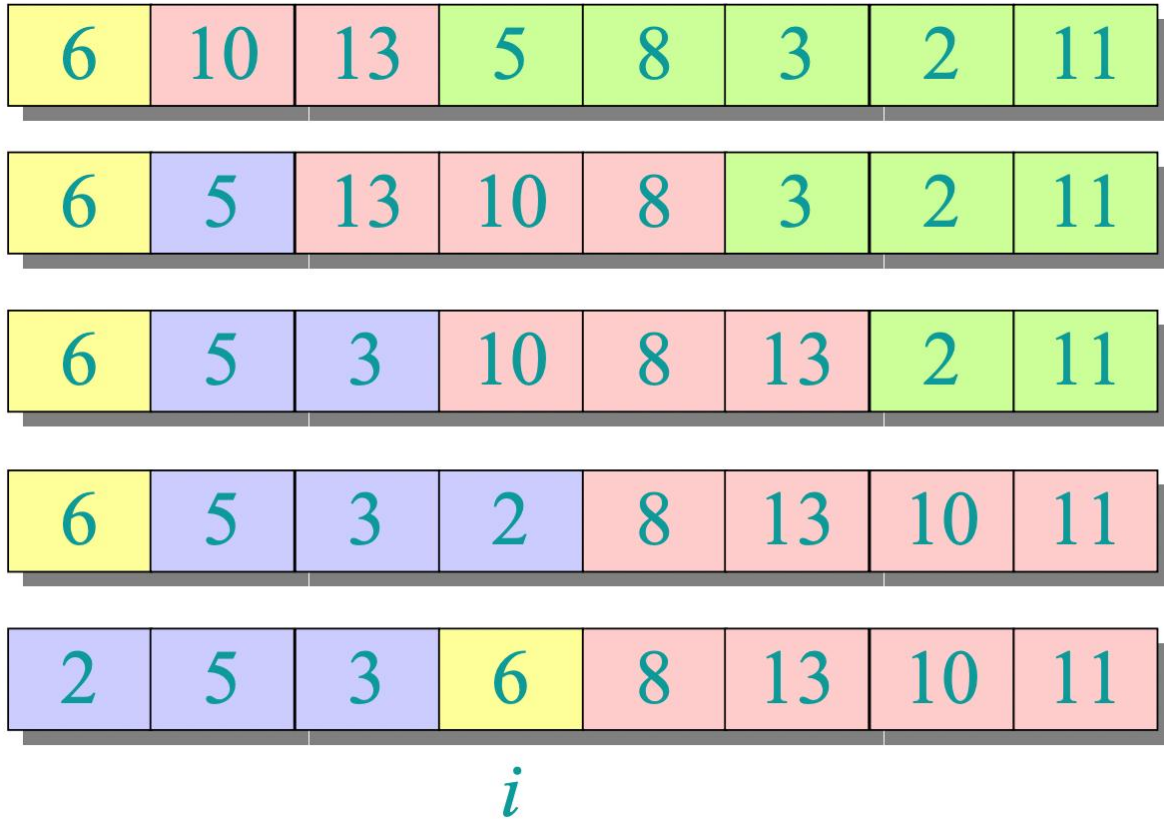
Invariant:



Example of partitioning

6	10	13	5	8	3	2	11
<i>i</i>	<i>j</i>						

Example of partitioning



QuickSort: Example

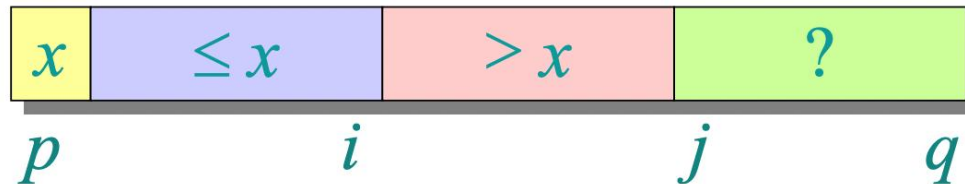
7	8	2	3	10
---	---	---	---	----

Running Time for Partition

```
PARTITION( $A, p, q$ )  $\triangleright A[p \dots q]$   
   $x \leftarrow A[p]$   $\triangleright \text{pivot} = A[p]$   
   $i \leftarrow p$   
  for  $j \leftarrow p + 1$  to  $q$   
    do if  $A[j] \leq x$   
      then  $i \leftarrow i + 1$   
           exchange  $A[i] \leftrightarrow A[j]$   
  exchange  $A[p] \leftrightarrow A[i]$   
  return  $i$ 
```

The running time for partition is $\Theta(n)$ since the pivot is compared with each element in the array.

Invariant:



Pseudocode for QuickSort

QUICKSORT(A, p, r)

if $p < r$

then $q \leftarrow \text{PARTITION}(A, p, r)$

 QUICKSORT($A, p, q-1$)

 QUICKSORT($A, q+1, r$)

Initial call: QUICKSORT($A, 1, n$)

Summary of QuickSort

- QuickSort is a randomized sorting algorithm that runs in expected time $\Theta(n \log n)$
- It is an in-place algorithm, so it does not need additional memory.
- It is simple to implement.

Sorting Overview

- Selection-Sort:
 - Very simple to implement.
 - In-place: Does not use additional memory.
 - For all inputs, runs in time $\Theta(n^2)$.

Sorting Overview

- Selection-Sort:
 - Very simple to implement.
 - In-place: Does not use additional memory.
 - For all inputs, runs in time $\Theta(n^2)$.
- Bubble-Sort:
 - Relatively easier to implement.
 - In-place.
 - Runs in time $O(n^2)$

Sorting Overview

- Selection-Sort:
 - Very simple to implement.
 - In-place: Does not use additional memory.
 - For all inputs, runs in time $\Theta(n^2)$.
- Bubble-Sort:
 - Relatively easier to implement.
 - In-place.
 - Runs in time $O(n^2)$
 - So it runs faster on nearly sorted arrays.
- Merge-Sort:
 - More difficult to implement.
 - Not in-place: Needs 2x extra memory for merging step.
 - Runs always in time $\Theta(n \lg n)$.

Sorting Overview

- Quick-Sort:
 - Simple to implement.
 - In-place: Does not use additional memory.
 - Runs in expected time $\Theta(n \log n)$, but has worst-case running time $\Theta(n^2)$.
- Counting-Sort:
 - Very simple to implement.
 - (Not in-place) It uses additional memory.
 - Runs in $\Theta(n + k)$, where k is the max. element in the array.
 - When $k=O(n)$, its running time is linear, i.e., $\Theta(n)$.